

1. Contexte et continuité avec la phase 1

La phase 1 a permis de poser les bases de l'application « MTL Vélo »: structure des pages, navigation, thème accessible et premier affichage tabulaire des données (compteurs, points d'intérêt, longueur du réseau cyclable). La phase 2 fait évoluer ce socle sur deux axes: l'introduction de la cartographie interactive dans la frontale, et la création d'une application dorsale (back-end) Node.js + Express avec une base SQLite, qui exposera les données via une API REST embryonnaire.

Réutilisation et évolution du travail de la phase 1: la frontale conserve sa structure de pages, sa navigation et son thème. Les boutons « Voir sur la carte » de la phase 1 (qui ouvraient un service externe) sont remplacés par des cartes interactives intégrées. Les filtres et la recherche restent en place mais s'enrichissent: le menu déroulant d'arrondissement est désormais synchronisé avec une carte des territoires. Le journal de démarche est poursuivi tout au long de la phase 2.

Les deux parcours de la phase 1 restent valorisés à égalité:

- **Parcours avec IA:** usage d'assistants génératifs pour le code frontal, le code dorsal, les requêtes SQL paramétrées et les tests, avec une vigilance accrue sur la revue critique - la complexité augmente, et un service web qui interroge une base est plus exposé aux erreurs (validation, fuites d'information, requêtes mal formées) que l'IA peut introduire.
- **Parcours sans IA:** appui sur la documentation officielle (Express, SQLite, Leaflet, Chart.js, etc.) et les références techniques, avec une démarche méthodique de résolution sur des problèmes nouveaux pour l'équipe - base de données, API REST, requêtes géospatiales.

Aucun parcours n'est privilégié: les deux permettent d'atteindre la note maximale (100 / 100). L'équipe peut conserver le parcours déclaré à la phase 1 ou en changer pour la phase 2; le nouveau choix est consigné dans **DEMARCHE.md** dès la première séance de la phase.

2. Objectifs pédagogiques de la phase 2

Objectifs communs aux deux parcours:

- **Intégrer une bibliothèque de cartographie** (Leaflet, OpenLayers, MapBox) pour afficher des données géospatiales en GeoJSON et superposer plusieurs couches.
- **Concevoir et implémenter une application dorsale** Node.js + Express (ou équivalent) qui sert la frontale et expose une première API REST.
- **Mettre en place une base de données** SQLite (ou équivalente) pour stocker les données de comptage horaire et y déléguer le filtrage.
- **Tracer les requêtes SQL paramétrées** et la validation minimale des entrées sur toute route exposée.
- **Tenir le journal de démarche** amorcé en phase 1, en distinguant clairement les sections frontale et dorsale.

Objectifs spécifiques au parcours avec IA:

- **Identifier les limites de l'IA** pour le code dorsal: hallucinations d'API ou de modules, signatures incorrectes, requêtes SQL non paramétrées, gestion d'erreurs absente. Documenter les hallucinations rencontrées.
- **Demander à l'assistant de générer des tests** (curl, Postman, Jest, Mocha) puis vérifier et corriger les tests proposés.

Objectifs spécifiques au parcours sans IA:

- **Naviguer la documentation** d'Express, SQLite, Leaflet et Chart.js de façon autonome pour intégrer ces technologies sans recourir à une IA.
- **Concevoir une stratégie de tests** manuels (curl, Postman) ou automatisés à partir de la documentation et des bonnes pratiques.

3. Tâches à réaliser

Données fournies

En plus des fichiers de la phase 1 (qui restent utilisés), pour ce livrable :

- **comptage_velo_2022.csv** à **comptage_velo_2026.csv** - un fichier par année, contenant les passages horaires par compteur (`date_heure`, `id_compteur`, `nb_passages`).
- **import_sqlite.py** - script Python d'exemple pour insérer ces données dans une base SQLite3. Vous pouvez l'utiliser tel quel ou le réimplémenter dans le langage de votre dorsale.

T1: Affichage cartographique du réseau cyclable

T1.1: Afficher une carte de Montréal sur la page « Réseau cyclable » à l'aide d'une bibliothèque de cartographie de votre choix.

T1.2: Tracer les pistes du fichier **reseau_cyclable.geojson** sur la carte avec une couleur correspondant à leur catégorie (REV, voie partagée, voie protégée, sentier polyvalent). Le mappage entre les attributs GeoJSON et les catégories est précisé en annexe.

T1.3: Afficher en bas de la carte un panneau récapitulatif dynamique: nombre de pistes visibles, longueur totale en kilomètres.

T1.4: Une fenêtre modale (overlay) doit s'ouvrir au clic sur une icône d'aide et présenter une légende des catégories de pistes.

T1.5: Implémenter le filtrage des pistes selon le type de voie (cases à cocher) et selon l'accessibilité 4 saisons (boutons), avec mise à jour en direct du panneau récapitulatif.

T2: Cartographie des compteurs et points d'intérêt

T2.1: Bouton « Carte » dans la vue « Statistiques »: ouvre une modale présentant tous les compteurs sur une carte. Le compteur cliqué initialement est mis en évidence visuellement.

T2.2: Bouton « Passages » sur chaque ligne de compteur: ouvre une modale permettant de choisir une période (date de début et date de fin) et affiche un graphique du nombre de passages à l'échelle journalière. Bibliothèques recommandées: Chart.js, D3 ou équivalents.

T2.3: Bouton « Carte » dans la vue « Points d'intérêt »: modale présentant les points d'intérêt actuellement listés, avec mise en évidence du point cliqué.

T3: Sélection interactive d'arrondissement

T3.1: Sur les vues « Réseau cyclable », « Statistiques » et « Points d'intérêt », l'utilisateur peut sélectionner un arrondissement de deux manières synchronisées: par clic sur le polygone correspondant sur une carte des territoires (alimentée par **territoires.geojson**) ou via le menu déroulant déjà en place depuis la phase 1.

T3.2: La sélection met le polygone en surbrillance et filtre effectivement les données affichées: pistes situées dans l'arrondissement, compteurs locaux, points d'intérêt locaux.

T4: Application dorsale et première API

T4.1: Démarrer un serveur web (Node.js + Express recommandé) qui sert la frontale à une URL telle que `http://localhost:8080/`. Aucune configuration CORS particulière n'est attendue ; respect de la politique d'origine identique.

T4.2: Implémenter les points de terminaison REST suivants:

- **GET /gti525/v1/compteurs/**: retourne la collection des compteurs.
- **GET /gti525/v1/compteurs/:id/passages?debut=YYYYMMDD&fin=YYYYMMDD**: retourne les passages agrégés par jour pour le compteur `:id`, filtrés sur la période demandée.

Les données proviennent d'une base SQLite peuplée à partir des fichiers `comptage_velo_*.csv`.

T4.3: Implémenter deux points d'accès préliminaires: **GET /gti525/v1/pistes** qui retourne le contenu de `reseau_cyclable.geojson`, et **GET /gti525/v1/pointsdinteret** qui retourne `poi.csv` converti en JSON.

T4.4: À partir de cette phase, toutes les requêtes SQL doivent utiliser des paramètres liés (jamais de concaténation de chaînes). La gestion d'erreurs côté serveur retourne un statut HTTP approprié et un objet `{ erreur: "..."}` , sans fuite de stack trace au client.

T5: Démarche réflexive et journal d'équipe

Le journal amorcé à la phase 1 est poursuivi. Les exigences augmentent à mesure que la complexité augmente: la dorsale et l'intégration cartographique introduisent des décisions techniques nouvelles qui méritent d'être tracées.

T5: Tâches communes aux deux parcours

- **T5.1:** Si l'équipe change de parcours pour la phase 2, mettre à jour **DEMARCHE.md** avec une brève justification.
- **T5.2:** Tenir le journal à jour avec au moins deux entrées critiques par membre sur des décisions de la phase 2 (par exemple: choix de la bibliothèque cartographique, choix entre Express et alternative, organisation des routes, schéma SQLite).

T5: Parcours avec IA

- **T5.A.1:** Mettre à jour **PROMPTS.md** avec les rubriques distinctes « Frontale », « Dorsale » et « Revue critique ». Chaque entrée suit la structure définie à **T3.A.1 (phase 1)** : prompt, outil et modèle, sortie obtenue, modifications humaines, et justification du jugement critique posé sur la sortie - pourquoi le code a été accepté, modifié ou rejeté. Pour le code dorsal, la justification mentionne explicitement les vérifications de sécurité (paramètres liés, validation des entrées, absence de fuite d'erreur).
- **T5.A.2:** Pour au moins une route de l'API, générer le code initial avec un assistant, puis tenir une trace explicite des modifications apportées: validation des paramètres, gestion d'erreurs, prévention des injections SQL. Cette trace est consignée dans **PROMPTS.md** sous « Revue critique ».
- **T5.A.3:** Tenir une liste des hallucinations rencontrées dans le journal: appels à des fonctions ou des modules qui n'existent pas, signatures incorrectes, mauvaises versions d'API. Au moins deux exemples documentés sont attendus.
- **T5.A.4:** Demander à l'assistant de générer au moins trois tests (curl, Postman, Jest ou Mocha) pour les routes implémentées, puis vérifier et corriger les tests proposés.

T5: Parcours sans IA

- **T5.B.1:** Étoffer **DEMARCHE.md** avec les rubriques « Frontale » et « Dorsale ». Pour chaque tâche significative, documenter le problème, les sources techniques consultées (documentation officielle, articles), au moins une alternative envisagée et la justification du choix retenu.
- **T5.B.2:** Indiquer dans les messages de commit toute référence externe directement utilisée (par ex. [ref: Express routing]) quand pertinent.
- **T5.B.3:** Concevoir et documenter une stratégie de tests: au moins trois tests (curl, Postman, Jest ou Mocha) pour les routes implémentées, avec brève justification du choix d'outil.
- **T5.B.4:** Tenir une réflexion sur les obstacles techniques particuliers à la phase 2 (par exemple: sécurité des requêtes SQL, gestion des erreurs HTTP, intégration cartographique) et leur résolution.

Important: le choix du parcours peut être révisé en cours de phase à condition d'être documenté dans **DEMARCHE.md**. Une équipe ne peut pas changer de parcours après la démonstration. La transparence est elle-même évaluée - voir Section 7.

4. Livrables

À la fin des quatre séances, chaque équipe remet:

- **Le code source complet** sur le dépôt Git de l'équipe (le même qu'en phase 1), avec un fichier **README.md** décrivant l'installation, le démarrage du serveur et la création de la base de données.
- **Le journal de démarche** à la racine du dépôt: **DEMARCHE.md** pour les deux parcours, accompagné de **PROMPTS.md** pour le parcours avec IA. Les deux fichiers conservent et étendent le contenu de la phase 1.
- **Un rapport** PDF de 5 pages maximum (hors page de titre, table des matières et références).
- **Une démonstration** en direct de 10 à 15 minutes lors de la dernière séance, incluant la frontale, l'API (test direct via curl ou Postman) et une présentation de l'organisation du code dorsal.

Procédure de soumission - gel de la version évaluée

Comme à la phase 1, la version évaluée est figée à l'aide d'un **tag Git annoté** pour permettre à l'équipe de continuer à travailler librement sur le dépôt après la date limite.

Au moment de la soumission, et au plus tard à la date limite, chaque équipe :

- crée un tag Git annoté nommé **phase2-submission** sur le commit à évaluer:
 - `git tag -a phase2-submission -m "Soumission phase 2"`
- pousse le tag vers le dépôt distant:
 - `git push origin phase2-submission`
- récupère le SHA complet du commit ciblé:
 - `git rev-parse phase2-submission`
- dépose sur Moodle, dans le formulaire de remise: le lien du dépôt, le nom du tag, le SHA complet du commit, ainsi que le rapport PDF.

Évaluation: le chargé de laboratoire évalue exclusivement la version du code identifiée par le tag. Toute divergence ultérieure entre le tag (côté dépôt) et le SHA déposé sur Moodle est traitée comme une tentative de modification après la date limite et entraîne la pénalité P7.

5. Contenu attendu du rapport

Le rapport est limité à cinq pages et doit répondre aux questions suivantes. La rubrique R3 a deux variantes selon le parcours déclaré dans **DEMARCHE.md** - choisir et traiter celle qui s'applique.

R1: Évolution de l'architecture frontale par rapport à la phase 1; choix de la bibliothèque cartographique et justification.

R2: Architecture dorsale: organisation des fichiers, routage, choix du cadriceil et de la base de données. Schéma simple de l'architecture client-serveur.

R3: Démarche professionnelle et réflexion critique.

- **Parcours avec IA:** trois prompts représentatifs (un frontal, un dorsal, un pour les tests) avec analyse de la sortie et des modifications apportées; identification des limites observées et des patrons de prompting efficaces.
- **Parcours sans IA:** trois décisions techniques représentatives (une frontale, une dorsale, une pour les tests) avec sources consultées, alternatives envisagées et justifications; obstacles surmontés et leçons.

R4: Sécurité minimale: comment l'équipe a traité la validation des entrées et la prévention des injections SQL. Exemples concrets dans le code.

R5: Répartition des tâches dans l'équipe et brève introduction et conclusion.

6. Grille d'évaluation

La phase 2 vaut 100 points, répartis en trois volets : démonstration, code et rapport, selon la grille du chargé de laboratoire.

Deux parcours, évalués à égalité:

- **Avec IA:** usage d'un assistant pour générer, améliorer ou analyser du code (frontal, dorsal, requêtes SQL, tests).
- **Sans IA:** développement entièrement manuel, appuyé sur la documentation et les références techniques.

Les deux visent la même compétence (pratique réflexive d'ingénierie) et permettent d'obtenir la note maximale.

Déclaration du parcours: inscrit dans **DEMARCHE.md** à la racine du dépôt dès la première séance de la phase. Le choix peut être révisé s'il est documenté; les critères C5 et R3 applicables sont alors ajustés.

6.1 Démonstration en direct (30 points)

Critère	Description	Points
---------	-------------	--------

D1 - Carte du réseau cyclable	La carte se charge, les pistes s'affichent avec leurs couleurs, le panneau récapitulatif est juste, la légende fonctionne.	6
D2 - Cartes secondaires	La carte des compteurs et celle des points d'intérêt fonctionnent, mise en évidence du point cliqué visible.	5
D3 - Graphique des passages	La modale du graphique des passages affiche des données cohérentes sur la période sélectionnée.	5
D4 - Sélection d'arrondissement	Synchronisation entre clic carte et menu déroulant, filtrage effectif des données dans chaque vue.	6
D5 - API dorsale	Les trois routes répondent correctement à des requêtes effectuées en direct (curl, Postman ou navigateur).	5
D6 - Maîtrise du code	Lors des questions, chaque membre est capable d'expliquer le code qu'il a contribué (frontale ou dorsale) et de justifier les choix techniques (et, le cas échéant, ce qui a été produit avec l'IA et comment il l'a évalué).	3

6.2 Code et dépôt Git (35 points)

Critère	Description	Points
C1 - Frontale et carte	Code modulaire, séparation des préoccupations, gestion d'erreurs sur les chargements asynchrones.	7
C2 - API et dorsale	Routes bien organisées, validation des paramètres, requêtes SQL paramétrées, codes HTTP appropriés et messages d'erreur sans stack trace.	10
C3 - Base de données	Base SQLite (ou équivalent) en place, schéma cohérent, données importées correctement.	5
C4 - Tests	Au moins trois tests fonctionnels présents et exécutables (script, fichier Postman, ou tests automatisés).	3
C5 - Journal de démarche	Tenue d'un journal de démarche réflexive selon le parcours choisi (voir ci-dessous). Les deux options sont équivalentes en points et en exigence.	10

C5 - Pratique réflexive et traçabilité:

- **Parcours avec IA: PROMPTS.md** à jour avec rubriques distinctes Frontale/Dorsale/Revue critique; au moins deux entrées critiques par membre sur la phase 2; au moins deux hallucinations documentées.
- **Parcours sans IA: DEMARCHE.md** à jour avec rubriques distinctes Frontale/Dorsale; pour chaque tâche: problème, sources, alternatives et justification ; au moins deux entrées par membre sur la phase 2.

Dans les deux cas, l'évaluation porte sur la qualité de la réflexion et la traçabilité des décisions.

6.3 Rapport (35 points)

Critère	Description	Points
R1 - Architecture frontale	Description claire de l'évolution depuis la phase 1, choix de la bibliothèque cartographique justifié.	6
R2 - Architecture dorsale	Description claire, schéma client-serveur présent, choix du cadriciel et de la BD justifiés.	8
R3 - Démarche professionnelle et réflexion critique	Section dédiée du rapport rendant compte du parcours choisi (voir ci-dessous). Mêmes attentes de profondeur dans les deux cas ; un constat superficiel est insuffisant.	12
R4 - Sécurité minimale	Validation des entrées et prévention des injections SQL, exemples concrets dans le code.	4
R5 - Répartition et présentation	Rôles précis, introduction et conclusion soignées, qualité de la mise en page.	5

R3 - Démarche et réflexion critique:

- **Parcours avec IA:** trois prompts représentatifs (frontal, dorsal, tests) avec analyse de la sortie et des modifications apportées; identification des limites observées; patrons de prompting efficaces. Une simple liste d'outils est insuffisante.
- **Parcours sans IA:** trois décisions techniques représentatives (frontale, dorsale, tests) avec sources consultées, alternatives envisagées et justifications; obstacles surmontés et leçons techniques. Une réponse vague est insuffisante.

6.4 Pénalités

- **P1 - Rapport mal rédigé:** qualité du français insuffisante (clarté, structure, syntaxe): **jusqu'à -10 points.**
- **P2 - Parcours avec IA:** code généré accepté tel quel, sans relecture ni adaptation, comme l'indiquent le dépôt et le journal de prompts: **-10 points.**

- **P3 - Parcours avec IA:** absence ou non-respect du journal **PROMPTS.md** malgré un usage manifeste de l'IA: **-10 points**.
- **P4 - Parcours sans IA:** absence ou non-respect du journal **DEMARCHE.md**, ou usage d'une IA non déclaré: **-10 points**.
- **P5 - Soumission non conforme:** tag `phase2-submission` absent du dépôt, pointant sur un commit différent du SHA déposé sur Moodle, ou créé après la date limite Moodle: **-15 points**.
- **P6 - Injection SQL:** requête SQL construite par concaténation de paramètres utilisateur (vulnérabilité d'injection présente dans le code livré) : **-10 points**.
- **P7 - Mauvaise gestion d'erreurs côté API:** le serveur plante sur une mauvaise requête, ou des stack traces sont renvoyées au client : **-5 points**.

Note: P3 et P4 sont symétriques selon le parcours choisi. P6 et P7 portent sur la qualité technique nouvelle introduite à la phase 2 et s'appliquent aux deux parcours.

7. Politique d'utilisation de l'IA et choix du parcours

Choix du parcours: libre et à la discrétion de l'équipe. À déclarer dans **DEMARCHE.md** dès la première séance de la phase, avec possibilité de révision si documentée. Les deux parcours sont équivalents et permettent d'atteindre 100 / 100.

Parcours avec IA - usages autorisés: génération de code, explication, refactorisation, génération de tests, suggestions de design, débogage assisté.

Parcours sans IA - ressources autorisées: documentation officielle (Express, SQLite, Leaflet, Chart.js, MDN, etc.), manuels, articles techniques, communautés (Stack Overflow pour comprendre, pas pour copier sans relecture). Les échanges en équipe et avec le chargé de laboratoire sont encouragés.

Interdictions (tous): soumettre du code sans relecture humaine, un rapport généré sans révision, du contenu copié d'une autre équipe ou session, ou un faux journal (**PROMPTS.md** / **DEMARCHE.md**).

Transparence: toute utilisation de l'IA doit être documentée. Les incohérences entre les déclarations et le code ou les commits sont pénalisées, dans les deux parcours.

Plagiat: le règlement de l'ÉTS s'applique. Le code est comparé entre sessions. La réutilisation de code d'une autre équipe ou d'une session antérieure est interdite.

Annexe 1 - Catégories de pistes cyclables

Les catégories de pistes cyclables sont déterminées à partir des attributs GeoJSON suivants:

Catégorie	Critères	Couleur suggérée
REV	REV_AVANCEMENT_CODE $\in$ { 'EV', 'PE', 'TR' }	#2AC7DD
Voie partagée	AVANCEMENT_CODE = 'E' et TYPE_VOIE_CODE $\in$ { 1, 3, 8, 9 }	#84CA4B
Voie protégée	AVANCEMENT_CODE = 'E', TYPE_VOIE_CODE $\in$ { 4, 5, 6 } et REV_AVANCEMENT_CODE $\notin$ { 'EV', 'PE', 'TR' }	#025D29
Sentier polyvalent	AVANCEMENT_CODE = 'E' et TYPE_VOIE_CODE = 7	#B958D9